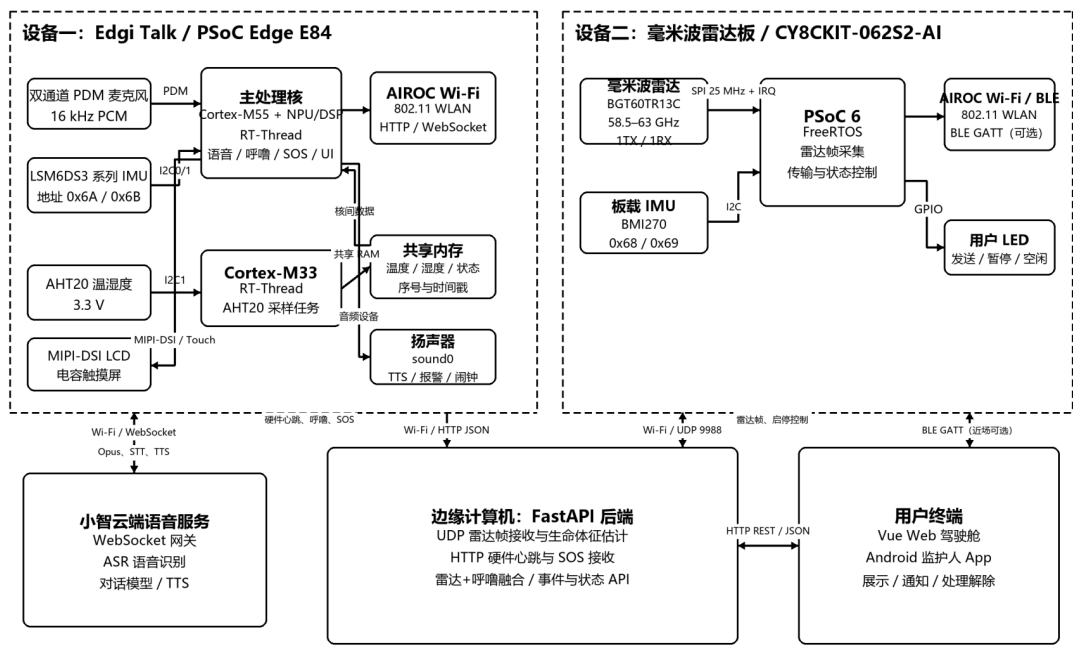


多模态非接触式睡眠监测系统

2026 年应用赛道作品设计报告（DOCX 可编辑版）

多模态非接触式睡眠监测系统硬件连接总框图



实线箭头表示当前主要数据链路；双向箭头表示控制与状态回传。

封面图 系统硬件连接与通信总框图

作品名称

多模态非接触式睡眠监测系统

摘要

本作品面向夜间睡眠看护与居家安全场景，设计并实现了一套多模态非接触式睡眠监测系统。系统以 PSOC Edge / Edgi Talk 开发板为边缘智能终端，结合毫米波雷达、PDM 麦克风、IMU、AHT20 温湿度传感器和 AIROC Wi-Fi，实现生命体征监测、呼噜识别、疑似睡眠呼吸暂停提示、睡眠环境评分、语音紧急求助和设备跌落告警。与摄像头看护相比，系统不采集图像，降低隐私压力；与可穿戴设备相比，床旁非接触式方案减少了佩戴、充电和摘除带来的使用门槛。

系统由两块主要真实硬件设备和软件平台组成：Edgi Talk / PSoC Edge E84 内部由 Cortex-M55 与 Cortex-M33 分工，M55 负责语音交互、共享麦克风采集、呼噜 int8 模型推理、IMU 紧急状态触发、SOS 显示与解除、闹钟和网络上报，M33 负责 AHT20 温湿度采样并通过共享内存交付环境数据；毫米波雷达板负责非接触采集胸腔微动相关原始数据，并通过 UDP 发送至后端。后端采用 FastAPI 汇聚真实硬件数据，维护设备在线状态、事件流、环境趋势和 active emergency 状态，并把雷达呼吸率与呼噜声证据进行窗口融合，生成 suspected_apnea 疑似呼吸暂停事件。Web 前端提供睡眠看护驾驶舱、看护预警中心和环境分析页面，Android 监护人 App 通过前台服务轮询后端，在后台或锁屏时接收紧急告警通知。

作品重点解决了真实样机联调中的麦克风资源抢占、Wi-Fi 配置持久化、报警解除闭环、传感器短时抖动误判、前端状态一致性和移动端后台提醒等工程问题。系统用于课程设计与看护辅助展示，不作为医疗诊断设备；疑似呼吸暂停、心率或呼吸异常等提示需要结合现场观察和专业设备复核。

第一部分 作品概述

功能与特性

系统围绕“睡眠过程少打扰、异常情况早发现、看护人员可处理”三个目标设计。核心功能包括：毫米波雷达非接触测量心率、呼吸率、距离和在床状态；边缘端呼噜检测模型持续识别环境中的呼噜声；后端融合雷达低呼吸窗口和呼噜变化，提示疑似睡眠呼吸暂停；小智语音识别“救命、需要帮助、喘不过气、摔倒了”等紧急话术并触发 SOS；IMU 检测开发板被打翻或跌落时也进入紧急状态；AHT20 提供温湿度，配合环境声音分贝形成睡眠环境评分。

- 真实硬件上报：正式接口统一为 /hardware/* 和 /emergency，避免模拟数据与真实数据混淆。
- 本地闭环处理：开发板出现 SOS 后本地报警、显示解除按钮，并可向后端同步解除状态。
- 多端联动：Web 和 Android App 均可看到关键事件，监护人可在任一端处理告警。

应用领域

作品适用于居家养老、独居老人夜间看护、养老机构巡护、慢病人群睡眠观察和宿舍/家庭睡眠环境辅助评估等场景。系统不依赖摄像头，不要求用户长期佩戴设备，适合放置在床旁进行低打扰监测。当用户无法主动操作手机或呼叫看护人员时，可通过语音求助或设备跌落触发紧急提示。对于长期睡眠观察，系统可以提供呼吸、心率、呼噜、温湿度和事件流的综合记录，为看护人员调整睡眠环境和观察异常趋势提供参考。

主要技术特点

- 边缘 AI：呼噜检测模型在 M55 侧以 int8 形式推理，降低原始音频长期上传需求。
- 共享麦克风：统一音频采集中心唯一持有 mic0，向唤醒词、小智语音和呼噜检测分发 PCM 数据，避免多线程抢占。
- 多传感器融合：雷达负责呼吸/距离，麦克风负责呼噜/求助语音，IMU 负责跌落触发，AHT20 负责环境评分。
- 真实后端闭环：FastAPI 维护设备在线状态、active emergency、事件流、历史趋势和告警解除记录。
- 移动端提醒：Android App 通过前台服务轮询后端，支持后台和锁屏时本地通知。

主要性能指标

指标	当前实现	说明
紧急语音触发	本地关键词 + 后端 critical 事件	支持“救命、需要帮助、喘不过气、摔倒了”等话术
呼噜检测周期	约 1 秒更新一次推理状态	2 秒滑动窗口，推理结果上报后端
环境上报	周期心跳	温度、湿度、sensor_ok、最近更新时间
Android 报警	约 5 秒轮询	同一事件去重，避免重复弹窗
报告提交限制	最终导出 PDF $\leq$ 50M	本版本为 DOCX 可编辑稿，便于修改后再导出

主要创新点

- 把小智语音从普通交互扩展为夜间紧急求助入口，并与本地 SOS 页面和远程告警闭环联动。
- 使用单一麦克风采集中心实现呼噜守护和小智语音并行，解决嵌入式音频资源抢占问题。
- 采用雷达低呼吸窗口 + 呼噜/恢复性声音证据的规则融合，减少单点雷达异常造成的误报。
- 用 Android 前台服务轮询后端替代近场蓝牙报警，使监护提醒不受蓝牙距离限制。

设计流程

设计流程分为需求分析、硬件分工、固件实现、后端接口、前端展示、Android 通知和真实联调七步。项目先确定非接触生命体征、呼噜、环境和 SOS 的核心需求，再将任务拆分到 Edgi Talk 的 M55/M33 双核和毫米波雷达板；随后实现 RT-Thread 多任务、跨核共享内存、共享麦克风、紧急状态机和正式硬件接口；最后通过 Web 驾驶舱、预警中心、环境分析页和 Android App 完成看护闭环。

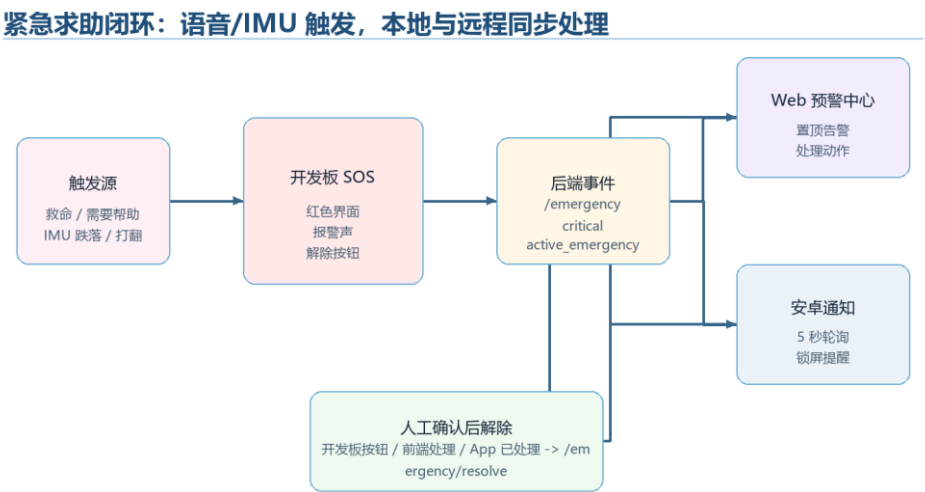


图 1-1 紧急求助闭环设计流程

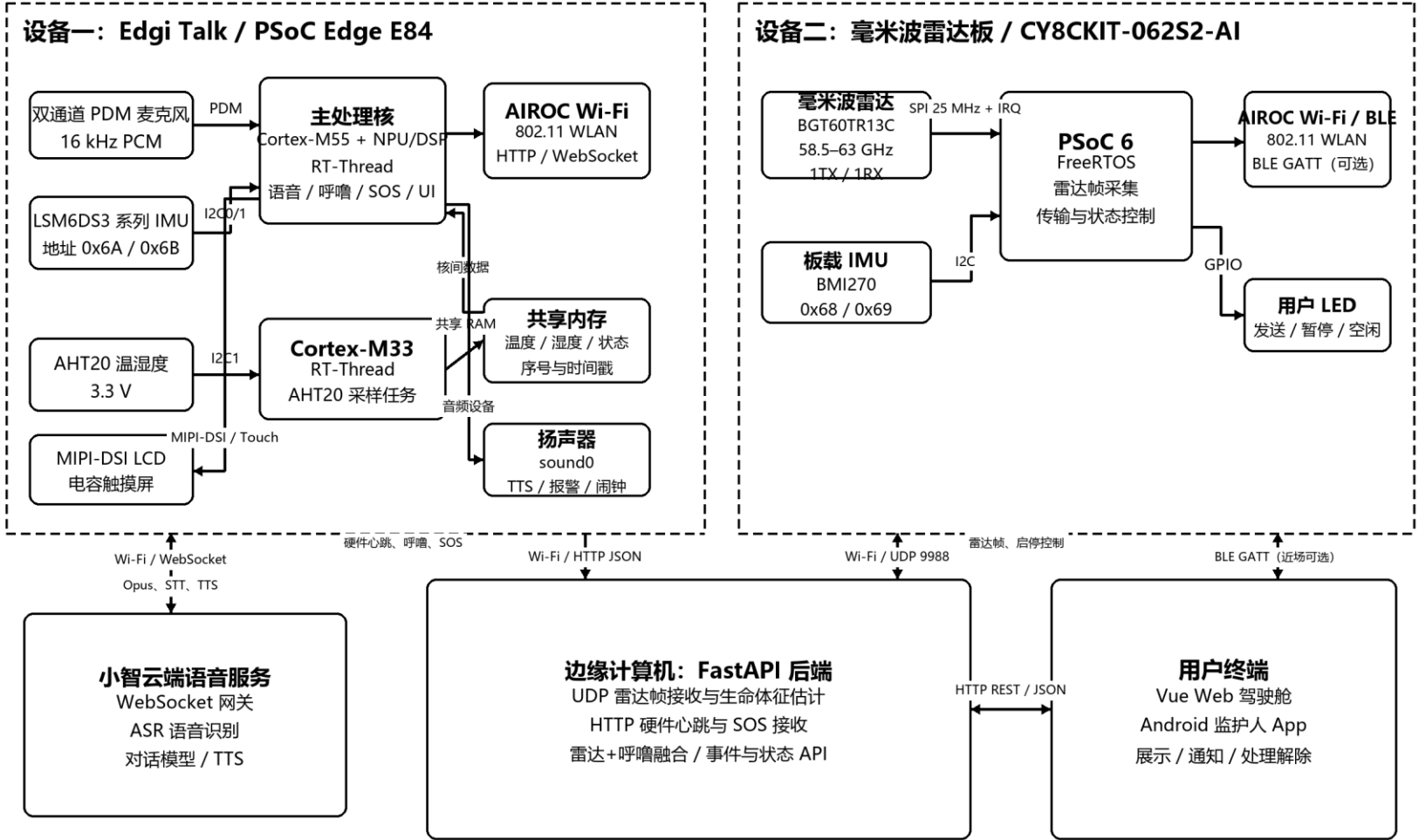
第二部分 系统组成及功能说明

整体介绍

系统硬件由两块主要设备构成。第一块是 Edgi Talk / PSoC Edge E84 床旁智能终端，同一芯片内包含 Cortex-M55 主处理核和 Cortex-M33 协处理核：M55 负责共享音频采集、小智语音、呼噜 INT8 推理、IMU 跌落判断、LVGL 界面、SOS 状态机和网络上报；M33 通过 I2C 周期采集 AHT20 温湿度，并经共享内存把温度、湿度、有效状态、序号和时间戳传递给 M55。第二块是 CY8CKIT-062S2-AI 毫米波雷达板，PSoC 6 通过 25 MHz SPI 与中断引脚采集 BGT60TR13C 雷达帧，通过 I2C 读取 BMI270，并使用 AIROC Wi-Fi/UDP 与本地后端交换雷达帧和启停控制。

设备通信采用分工明确的协议：Edgi Talk 通过 Wi-Fi/HTTP JSON 上报呼噜、环境心跳、语音或 IMU 紧急事件，通过 WebSocket 与小智云端交换 Opus 音频、STT 和 TTS；雷达板通过 Wi-Fi/UDP 9988 发送雷达帧并接收启停指令；FastAPI 后端通过 REST/JSON 向 Vue Web 和 Android App 提供状态、时间线、事件和处理接口。雷达 BLE GATT 作为近场查看的可选旁路，不承担远程告警主链路。

多模态非接触式睡眠监测系统硬件连接总框图



实线箭头表示当前主要数据链路；双向箭头表示控制与状态回传。

图 2-1 硬件资源连接、设备关系与通信协议总框图

主要硬件接口与通信链路

连接对象	硬件接口/协议	数据内容	用途
PDM 麦克风 → M55	PDM / mic0, 16 kHz	双通道 PCM	唤醒词、小智语音、呼噜推理共享输入
LSM6DS3 → M55	I2C0/1, 0x6A/0x6B	三轴加速度、自由落体状态	设备跌落与无法呼救场景 SOS
AHT20 → M33	I2C1, 3.3 V	温度、相对湿度	2 秒周期环境采样
M33 → M55	片上共享内存	温湿度、状态、序号、时间戳	跨核环境数据交付
BGT60TR13C → PSoC 6	SPI 25 MHz + IRQ	1TX/1RX 雷达帧	距离、呼吸率和心率估计输入
BMI270 → PSoC 6	I2C, 0x68/0x69	三轴加速度与运动量	雷达板移动/静止状态
Edgi Talk → 后端	Wi-Fi / HTTP JSON	呼噜、环境、SOS、解除	设备状态与紧急事件闭环
雷达板 ↔ 后端	Wi-Fi / UDP 9988	雷达帧、启停命令	实时采集与后端生命体征处理

硬件系统介绍

2.2.1 硬件整体介绍

Edgi Talk 端采用异构双核分工。Cortex-M55 运行主要 RT-Thread 应用和边缘 AI，直接连接 PDM 麦克风、LSM6DS3、MIPI-DSI 显示与触摸、扬声器和 AIROC Wi-Fi；Cortex-M33 运行精简的 AHT20 采样任务。M33 不作为独立环境板，而是 Edgi Talk/PSoC Edge E84 内部协处理核，两核通过固定共享内存数据结构交换环境样本，避免传感器任务影响 M55 的音频实时性。

雷达端以 PSoC 6 为控制核心，BGT60TR13C 负责非接触胸腹微动采集，BMI270 提供板体运动参考，FreeRTOS 队列在 radar_task、config_task 和 udp_server 之间传递帧与命令。当前真实演示使用 Wi-Fi/UDP 作为雷达主链路，用户 LED 指示发送、暂停和空闲状态。

2.2.2 机械设计介绍

当前样机以开发板和传感器模块组合为主，机械结构重点在于摆放位置与操作可见性。Edgi Talk 放置在床旁，屏幕朝向看护或用户，便于看到 SOS、呼噜状态和闹钟界面；雷达板朝向人体胸腹区域，保证非接触采集稳定；AHT20 传感器通过 Edgi Talk 扩展接口放置在床旁空气流通处，避免靠近发热源或出风口。后续可进一步设计一体化外壳，把显示屏、雷达支架、麦克风开孔和温湿度通风口集成到床头设备中。

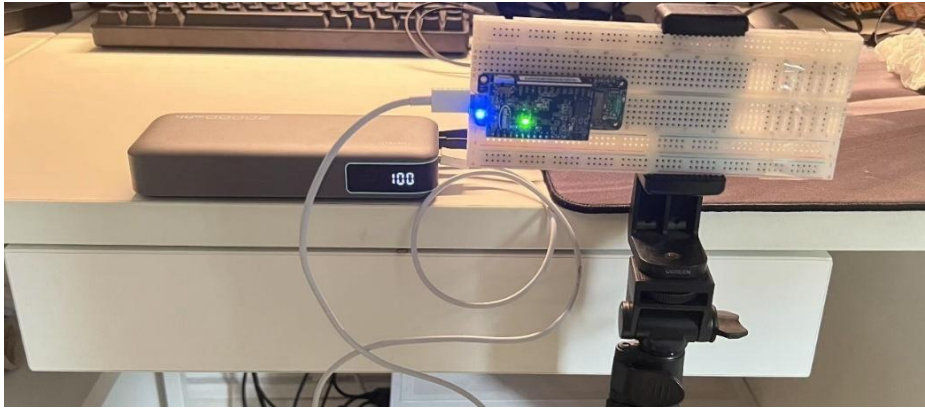


图 2-2 开发板与雷达实物连接示意

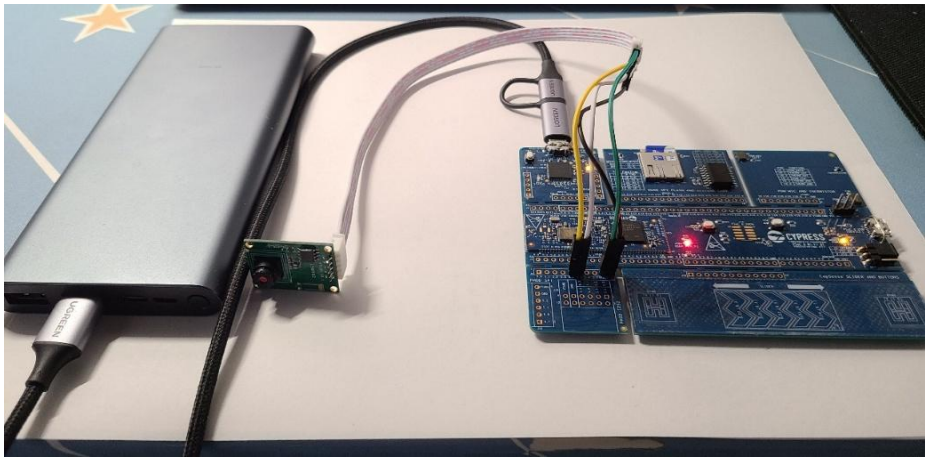


图 2-3 雷达模块测试摆放场景

2.2.3 电路各模块介绍

电路部分重点使用开发板自带的 PDM 麦克风、AIROC Wi-Fi、IMU、I2C 传感器接口和雷达模块接口。报告中保留与当前作品直接相关的原理图，便于说明信号输入、输出和通信链路。

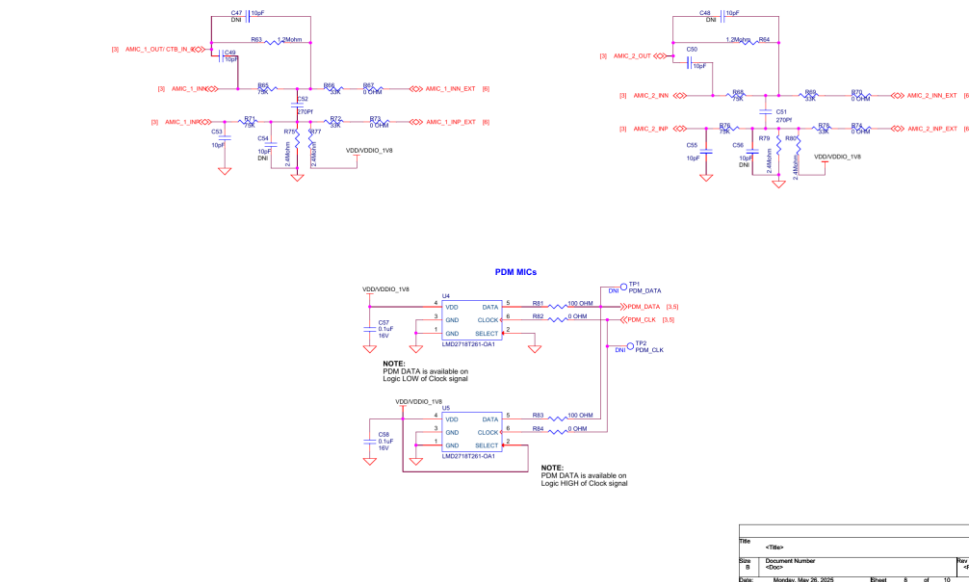


图 2-4 PDM 麦克风原理图，用于小智语音和呼噜检测输入

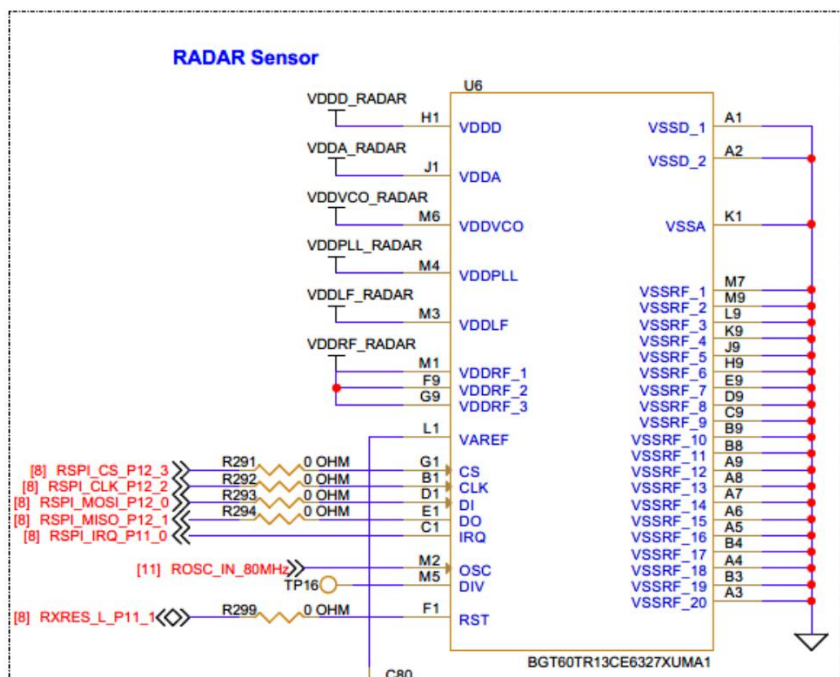
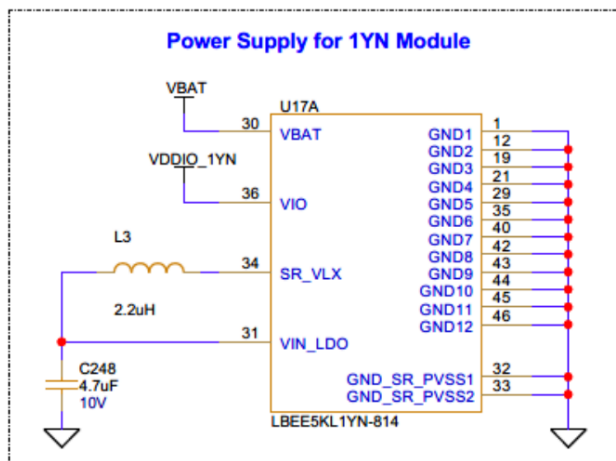


图 2-5 毫米波雷达传感器接口原理图



1YN module power supply schematic

图 2-6 AIROC Wi-Fi 模块供电与接口原理图

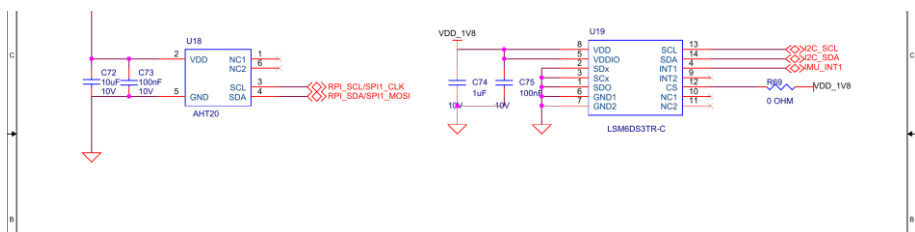


图 2-7 AHT20 与 IMU 相关原理图

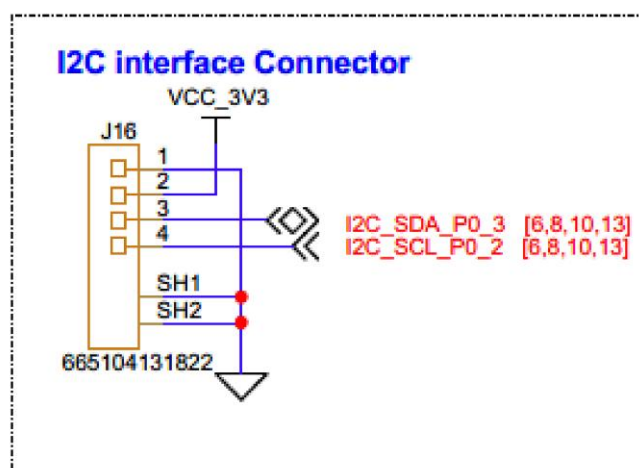


图 2-8 I2C 接口连接器原理图

该 I2C 扩展接口对应 J16.3/P0[3] 作为 I2C_SDA，J16.4/P0[2] 作为 I2C_SCL，逻辑电平为 3.3 V，可用于 AHT20 等外接 I2C 传感器扩展。

软件系统介绍

2.3.1 软件整体介绍

软件架构按端、边、云三层划分。端侧包括 Edgi Talk 的 M55/M33 RT-Thread 固件、雷达板 FreeRTOS 固件，以及 Vue Web 和 Android 用户终端；边缘侧是运行在本地计算机上的 FastAPI 服务，负责 UDP/HTTP 接入、雷达生命体征计算、统一设备状态、雷达与呼噜融合、事件去重和 REST API；云侧只承担小智语音的 WebSocket 会话、ASR、对话理解和 TTS，不保存雷达原始帧，也不直接决定本地 SOS。

多模态睡眠看护系统端—边—云软件架构

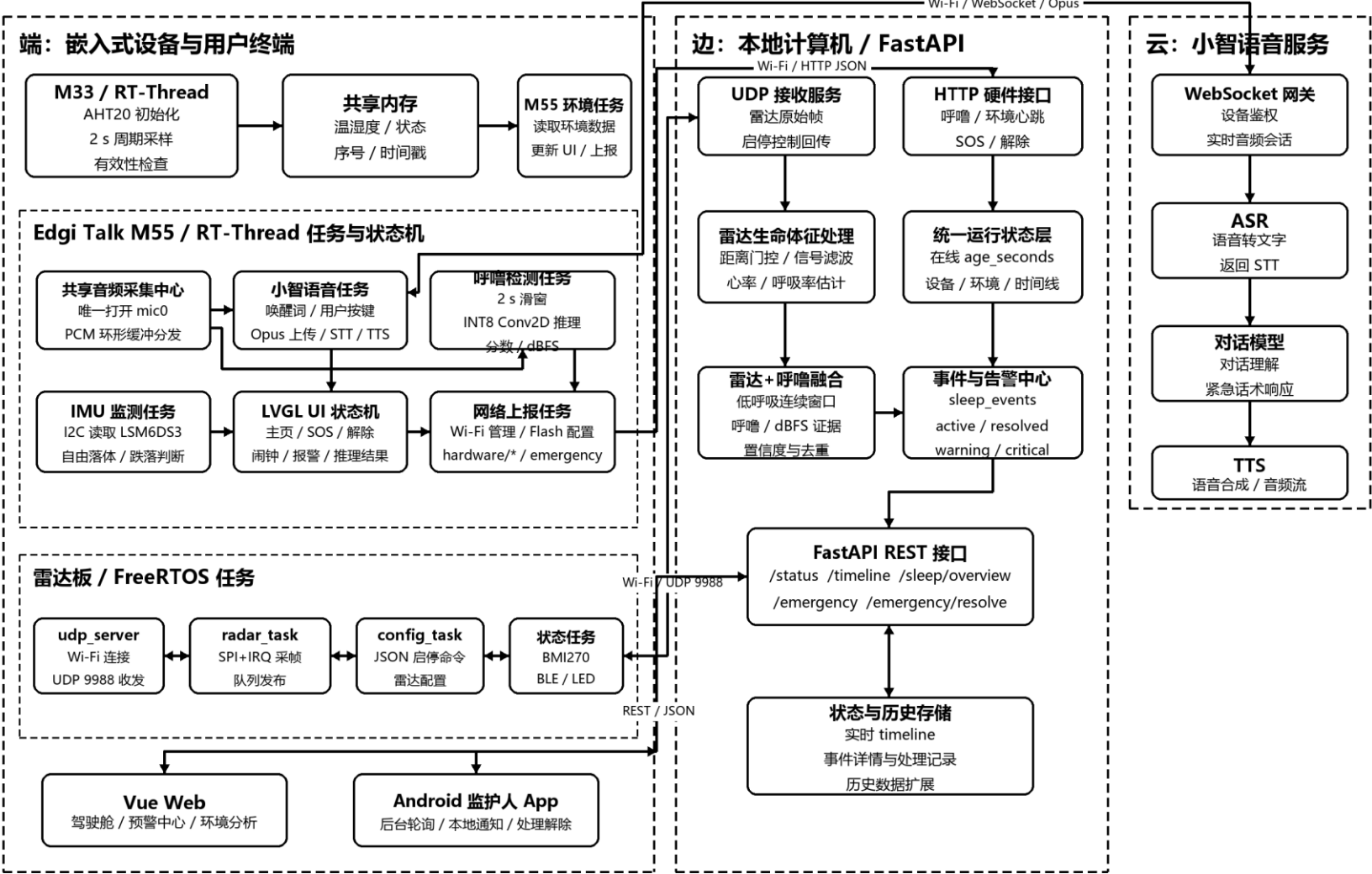


图 2-9 端—边—云软件架构与任务关系

端边云执行域职责

执行域	操作系统/框架	关键任务或模块	主要输出
Edgi M33	RT-Thread	AHT20 初始化、2 秒采样、有效性校验、共享内存写入	温湿度与传感器状态
Edgi M55	RT-Thread	音频中心、小智语音、呼噜 INT8、IMU、UI、报警、网络上报	呼噜/环境心跳、SOS、解除
雷达板	FreeRTOS	udp_server、radar_task、config_task、BMI270 状态、BLE/LED	雷达帧与板体状态
边缘计算机	FastAPI/Python	雷达处理、设备状态、融合判断、事件中心、REST API	生命体征、时间线、预警事件
小智云端	WebSocket/ASR/TTS	音频会话、语音识别、对话响应、语音合成	STT 文本与 TTS 音频

真实后端接口与数据对象

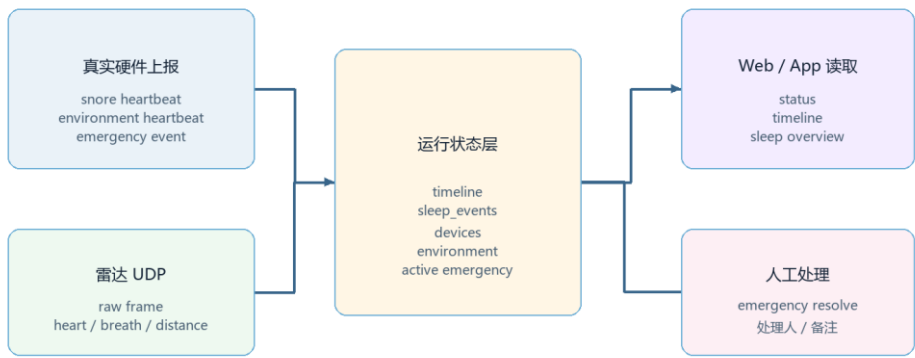


图 2-10 真实后端接口与数据对象

接口/链路	调用端	功能
/hardware/snore-session/start	M55 呼噜守护	标记呼噜检测开始
/hardware/snore-heartbeat	M55 呼噜检测任务	上报 snore_score、dbfs、detected
/hardware/environment-heartbeat	环境上报任务	上报温度、湿度和 sensor_ok
/emergency	语音或 IMU SOS	创建 critical 紧急事件
/emergency/resolve	开发板/Web/App	解除紧急状态并记录处理来源
UDP 雷达帧	雷达板	发送原始雷达帧供后端解析

2.3.2 嵌入式端软件架构

M33 固件启动后创建 env_m33 线程，初始化 AHT20/AHT10 兼容驱动，以 2 秒周期读取温湿度，对范围进行有效性检查，并把定点化后的 x10 数值、valid、status、seq 和 updated_ms 写入

共享内存。M55 环境任务读取同一数据结构，更新开发板主页，同时通过 /hardware/environment-heartbeat 上报后端。该跨核分工把低速环境采样与音频/显示主任务隔离。

M55 固件的关键模块是共享麦克风采集中心。它唯一打开 mic0，持续读取 16 kHz 双通道 PCM，并通过非阻塞环形缓冲分发给唤醒词、小智语音上传和呼噜检测消费者。待机时唤醒词和呼噜消费者常驻；唤醒或按键后开启 Opus 语音上传并暂停唤醒词推理，呼噜消费者保持运行；播放 TTS、闹钟或报警时只抑制呼噜告警，采集线程不关闭。某个消费者处理慢时只丢弃自己的旧数据，不能阻塞其他消费者。

IMU 任务轮询 I2C0/i2c1 上 0x6A、0x6B 地址的 LSM6DS3 系列器件，结合自由落体状态位和加速度幅值判断设备跌落；一旦触发，统一进入 SOS 状态机，显示解除按钮、播放本地报警并上报 critical 事件。UI、报警、闹钟和网络请求均通过状态变量与任务边界解耦，避免触摸回调中直接删除线程或执行长时间阻塞操作。

雷达固件在 FreeRTOS 下由 udp_server 建立 Wi-Fi 和 UDP 9988 套接字，radar_task 使用 SPI 25 MHz 与 IRQ 采集 BGT60TR13C 帧，config_task 解析 radar_transmission 启停 JSON，BMI270 状态任务负责板体运动信息，BLE/LED 任务提供近场摘要和本地状态指示。雷达原始帧不在板端执行复杂生命体征算法，而是通过队列交给 UDP 任务发送至边缘计算机。

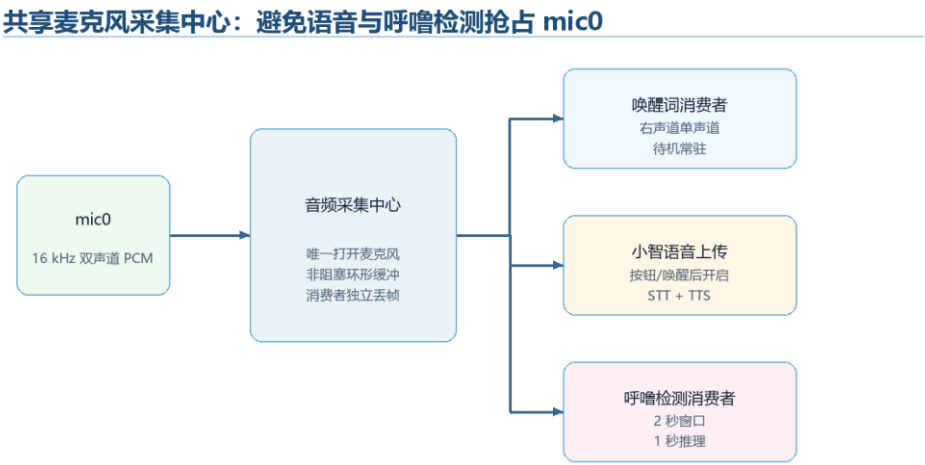


图 2-11 共享麦克风采集中心流程

2.3.3 边缘与云端功能

边缘后端同时接收雷达 UDP 帧和 Edgi HTTP 心跳。雷达数据经过距离门控、信号滤波和生命体征估计得到心率、呼吸率与距离；HTTP 数据更新呼噜、dBFS、环境、语音和紧急状态。统一状态层使用 age_seconds 判断在线状态，事件中心维护 active/resolved、severity、source 和 details，并通过 /status、/timeline、/sleep/overview 提供给 Web 与 Android。

疑似呼吸暂停融合模块运行在边缘端。该模块观察 30 到 60 秒窗口中的雷达呼吸率、目标在床状态、雷达在线状态和呼噜强度变化。当目标在床且雷达数据连续低呼吸或丢失，并伴随呼噜增强或恢复性声音证据时，生成 suspected_apnea 事件；人不在床、雷达离线或只有单点异常时不触发该事件。小智云端只负责语音识别、对话响应与语音合成，本地关键词和 SOS 上报仍由 M55 完成，即使云端短时不可用也不影响 IMU 紧急触发。

疑似呼吸暂停融合：雷达低呼吸 + 呼噜/恢复性声音证据

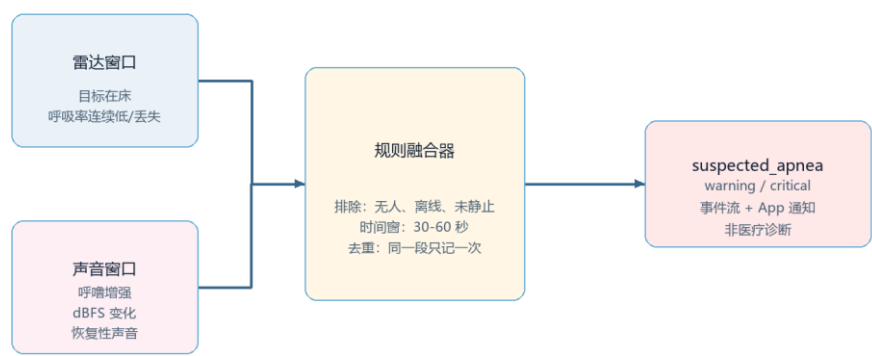


图 2-12 雷达 + 呼噜融合判断疑似呼吸暂停

第三部分 完成情况及性能参数

整体介绍

当前系统已形成可演示的真实硬件闭环：M55 小智板可联网并向后端上报呼噜、环境和 SOS 事件；雷达板可向后端发送生命体征相关数据；前端可展示驾驶舱、预警中心和环境分析；Android App 可填写后端地址并接收紧急通知。



图 3-1 系统实物全局连接示意（一）

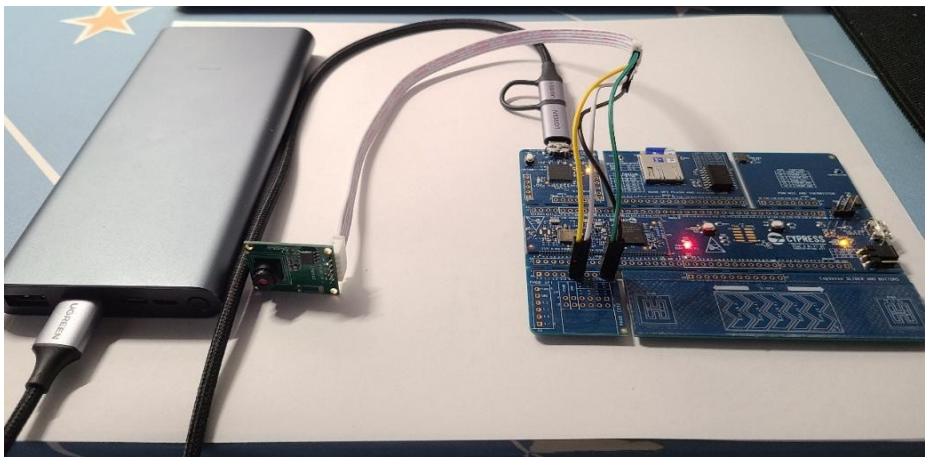


图 3-2 系统实物全局连接示意（二）

工程成果

3.2.1 机械成果

样机采用模块化摆放方式完成真实联调，M55 屏幕用于床旁状态显示和触摸操作，雷达板通过支架或桌面固定朝向人体区域，环境传感器放置在床旁空气流通处。后续可以将三类模块整合为一体式床头看护设备。

3.2.2 电路成果

电路成果主要体现在多开发板资源复用和接口打通：PDM 麦克风输入支撑语音与呼噜检测，IMU 支撑跌落 SOS，AHT20 通过 I2C 提供温湿度，AIROC Wi-Fi 支撑 HTTP/UDP 通信，雷达模块提供非接触生命体征输入。

3.2.3 软件成果

软件成果包括 M55 固件、雷达板固件、真实后端、Vue 前端和 Android App。M55 固件包含 SOS 页面、解除按钮、闹钟页面、呼噜推理显示和环境上报；后端提供正式硬件接口和睡眠概览；前端页面保持简洁，以关键状态、趋势图和事件流为主；Android App 负责后台监听和本地通知。

以下 Web 界面截图来自本项目 Vue 前端实际运行页面，包含项目首页、睡眠看护驾驶舱、看护预警中心和睡眠环境分析。截图中的侧边栏、卡片、趋势图、SOS 处理区和设备状态均由前端代码渲染，便于比赛评审直观看到作品的软件完成度。

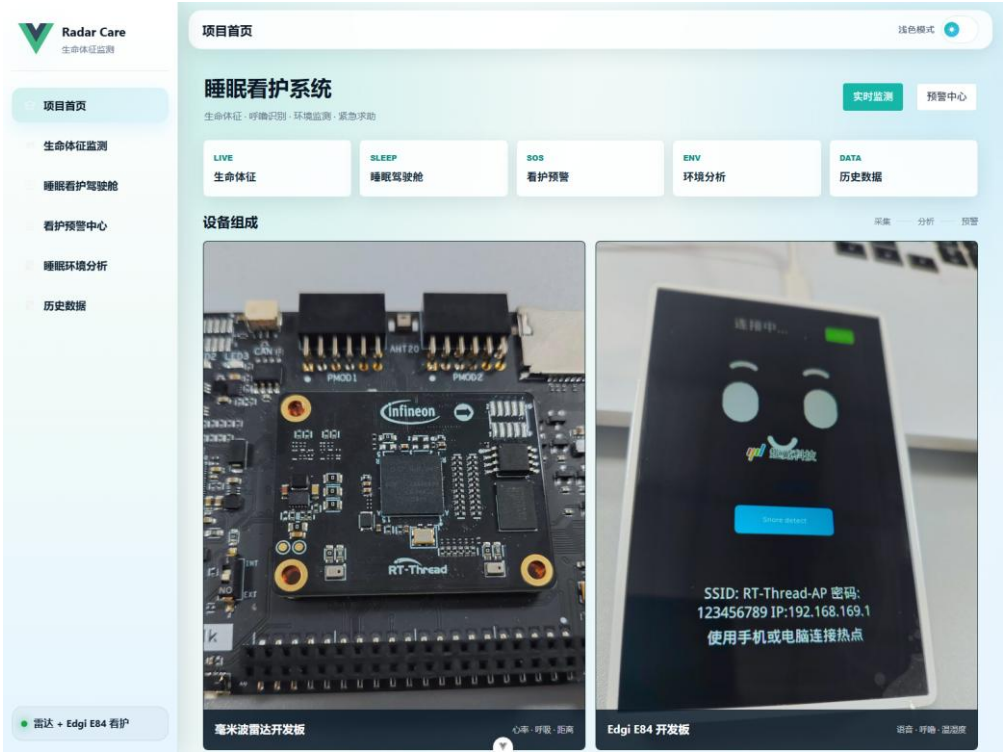


图 3-3 Web 项目首页真实运行界面

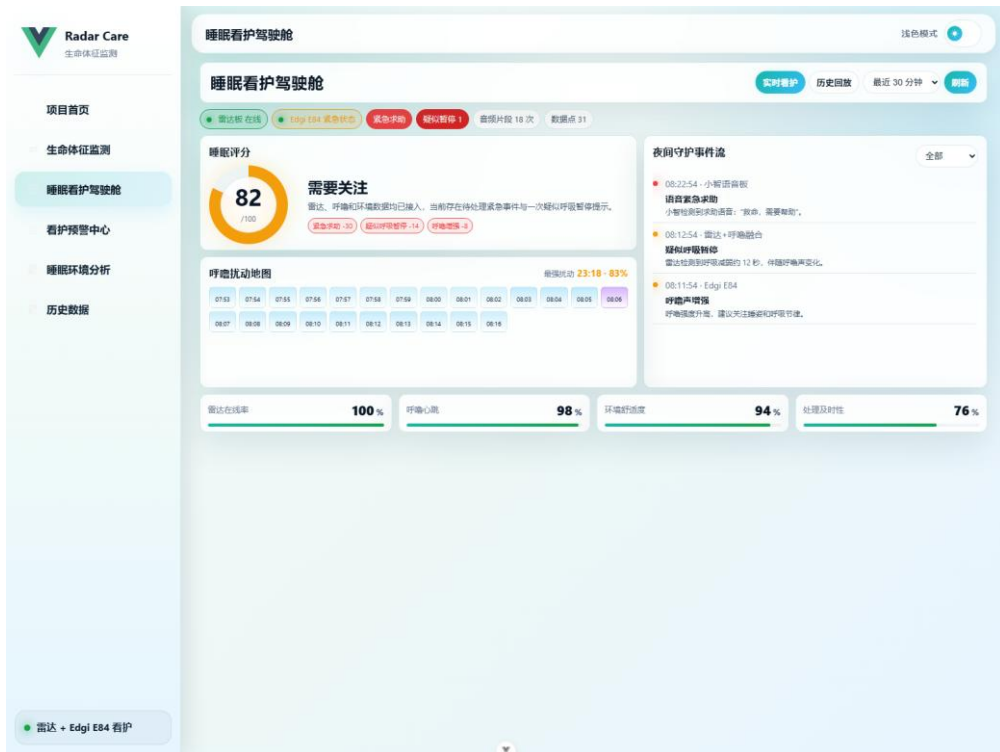


图 3-4 Web 睡眠看护驾驶舱真实运行界面

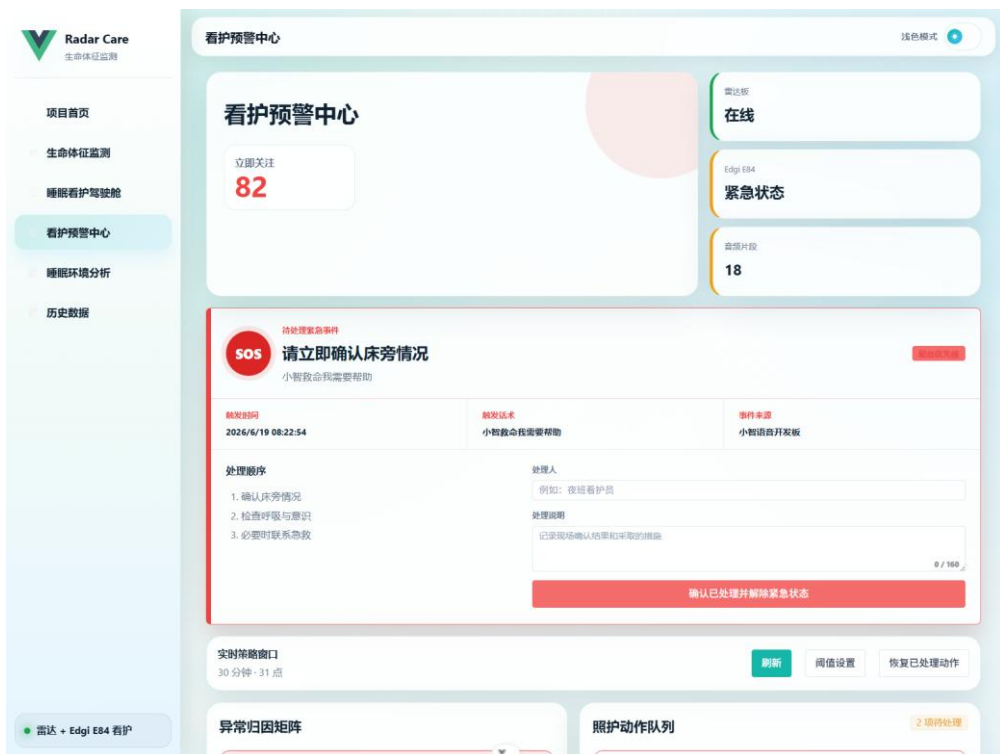


图 3-5 Web 看护预警中心真实运行界面



图 3-6 Web 睡眠环境分析真实运行界面

Android Guardian App: Sleep Alerts



图 3-7 Android 睡眠监护报警 App 后端绑定与告警处理界面

特性成果

测试项	测试方法	实现结果
语音 SOS	说出“救命/需要帮助”等关键词	开发板进入 SOS，后端生成 critical 事件，Web/App 同步提醒
IMU SOS	串口命令或低高度软垫打翻测试	触发设备跌落紧急事件，并显示手动解除按钮
呼噜检测	播放或采集呼噜样本	开发板显示推理结果，后端收到 snore-heartbeat
环境监测	AHT20 周期采样	前端显示温湿度、分贝、环境评分和建议

测试项	测试方法	实现结果
疑似呼吸暂停	雷达低呼吸窗口 + 呼噜变化模拟/实测	后端生成 suspected_apnea 事件并进入预警中心
Android 通知	后端出现 active critical 事件	手机约 5 秒内收到本地通知，同一事件去重

指标	结果/参数	备注
前端构建	npm run build 通过	用于 Web 部署
后端运行	realtime_radar_processing.py	真实硬件数据优先
紧急事件严重度	critical	语音 SOS 和 IMU SOS 均使用最高优先级
疑似呼吸暂停严重度	warning / critical	按持续时间和融合证据调整
App 轮询周期	5 秒	适合课程设计和局域网/公网演示

关键需求与设计约束

本作品的设计对象不是单一传感器演示，而是夜间看护场景中的完整工作流。夜间睡眠具有低交互、低照明、用户反应慢和异常发生突然等特点，因此系统必须在用户不主动操作的情况下持续观察关键状态；当出现求助、跌落或疑似呼吸暂停时，又必须把复杂数据转化为看护人员能立即理解的告警。基于这一场景，系统把需求拆分为四类：第一类是生命体征与在床状态，主要由毫米波雷达提供；第二类是声音线索，包括呼噜强度、环境分贝和语音求助，主要由 Edgi Talk M55 的麦克风提供；第三类是环境舒适度，由 AHT20 温湿度和声级趋势共同衡量；第四类是紧急处理闭环，包括本地报警、Web 置顶、手机通知和手动解除。

工程实现中还存在明显约束。开发板资源有限，不能长期保存大段原始音频，也不能让多个线程同时打开同一个麦克风设备；雷达呼吸预测容易受到人体姿态、距离、板子晃动和环境反射影响，不能把单点异常直接等同于医学意义的呼吸暂停；Android 手机在后台接收报警时，如果完全依赖蓝牙近场连接，会受到距离和系统后台策略影响。因此本作品采用“边缘端先提取特征、后端做状态融合、前端只展示关键信息、App 用后端地址接收远程报警”的方案，把系统复杂度放在可解释、可联调和可演示的范围内。

另一个重要约束是误报风险。语音关键词如“救命、需要帮助、喘不过气”具有较高紧急程度，系统应立即触发 critical 告警；但呼噜、雷达低呼吸、温湿度不适等现象更适合形成分级提示。为避免把普通翻身、短时静音或雷达抖动误判为严重事件，后端对疑似呼吸暂停采用窗口判断和证据叠加：需要目标在床、雷达在线、呼吸连续减弱或丢失，并结合呼噜增强或恢复性声音变化才生成事件。这样的设计使作品更接近看护辅助设备，而不是把算法输出直接变成结论。

边缘端固件实现细节

Edgi Talk M55 固件是本作品交互和边缘 AI 的核心。固件在 RT-Thread 下拆分为 Wi-Fi 管理、后端配置、共享音频采集、呼噜检测、小智语音、IMU 跌落检测、环境上报、屏幕 UI、报警播放和闹钟等任务。Wi-Fi 管理负责首次 AP 配网、配置保存和重启后自动连接；后端配置保存电脑 IP 与端

口，当前真实环境统一指向 192.168.31.236:8081；屏幕 UI 负责主页、呼噜守护状态、SOS 状态、解除按钮和闹钟响铃界面。各任务通过轻量状态变量和互斥保护协同，尽量避免在 UI 回调中执行耗时网络请求或音频操作。

共享麦克风是 M55 固件中最关键的改造点。早期版本中，小智语音、唤醒词和呼噜检测分别打开 mic0，在连续启停或快速按键时容易出现抢占、阻塞和线程断言。当前版本改为单一采集中心：采集中心唯一持有 mic0，持续读取 16 kHz 双声道 PCM，再把数据分发给不同消费者。唤醒词和呼噜模型使用右声道单声道数据，小智语音保留双声道数据进入原有上传链路。每个消费者都有独立的非阻塞环形缓冲，当某个模型推理过慢时只丢弃自己的旧数据，不影响麦克风采集，也不影响其他消费者继续工作。

呼噜检测模型采用 int8 量化后的 C 头文件部署在固件中，避免运行时加载 h5 或 tflite 文件。模型输入来自 2 秒单声道滑动窗口，每约 1 秒执行一次推理，输出 snore_score、snore_level 和 detected 状态。为防止本机报警音、闹钟音和 TTS 被误识别成呼噜，固件在扬声器播放期间设置短时抑制窗口：采集仍然持续，但呼噜告警与事件上报会被降权或暂停。这样可以保证小智语音、呼噜守护和报警提示同时存在时，系统仍然稳定。

SOS 状态机包含语音触发和 IMU 触发两条入口。语音入口在小智 STT 文本分支中进行本地关键词匹配，命中“救命、帮帮我、需要帮助、喘不过气、胸口痛、摔倒了”等短语后立即显示 SOS 页面并上报 /emergency。IMU 入口用于模拟病人无法呼救时把床旁开发板打翻到地上的场景，检测到跌落或异常姿态后同样进入 SOS。两条入口共享本地报警、红色紧急界面、解除按钮和后端事件格式，解除时通过 /emergency/resolve 同步到后端，使前端和 App 不会继续显示 active emergency。

雷达板固件则保持原始雷达数据链路简洁：Wi-Fi 连接成功后等待后端发送启用命令，随后通过 UDP 发送雷达帧，后端解析后得到心率、呼吸率、目标距离和在线状态。为便于真实联调，曾加入板子静止判定和 LED 指示；在当前测试阶段可按需要临时关闭静止门控，优先验证雷达数据连续性。Edgi Talk 内部的 M33 环境任务负责初始化 AHT20/AHT10 兼容传感器，周期采样温度和湿度，并通过共享内存交给 M55 上报。为了减少串口干扰，温湿度和呼噜检测的高频日志已经从正式演示版本中收敛。

真实后端与数据闭环

后端采用 FastAPI 实现，定位为真实硬件数据汇聚层，而不是简单的数据转发层。后端接收三类数据：一是雷达板 UDP 数据，经过解析后形成心率、呼吸率、距离、目标存在和雷达在线状态；二是 Edgi/M55 通过 HTTP 上报的呼噜会话、呼噜心跳、环境心跳和紧急事件；三是 Web 或 Android 端发送的紧急解除请求。后端把这些数据统一写入内存状态和事件列表，并通过 /status、/timeline、/sleep/overview 等接口提供给前端和 App。

设备在线判断采用最近心跳时间而不是简单连接状态。雷达、呼噜、环境和语音服务都有独立 `age_seconds` 字段，前端可以区分“整块 Edgi 在线但某个功能暂停”“雷达在线但未静止”“Edgi 在线但 AHT20 传感器异常”等情况。紧急状态期间，后端还加入短时宽限逻辑，避免报警声、网络抢占或任务切换导致呼噜/环境心跳短时间缺失时前端立即显示掉线。这个细节对实际演示非常重要，因为 SOS 触发后系统同时播放声音、刷新 UI、上报事件和等待解除，短时间资源占用会明显增加。

事件模型统一使用 `event_type`、`severity`、`title`、`message`、`source`、`timestamp`、`status` 和 `details` 等字段。语音 SOS 和 IMU SOS 统一为 `critical` 级别；疑似呼吸暂停根据持续时间和证据强度分为 `warning` 或 `critical`；呼噜增强、环境不舒适和设备离线则按影响程度进入事件流。Android App 和 Web 预警中心都只依赖同一套事件模型，因此开发板、前端和手机端可以看到一致的告警内容。处理事件时，`/emergency/resolve` 会记录处理来源、处理人和备注，前端刷新后 `active emergency` 消失，历史事件保留已处理状态。

疑似呼吸暂停融合算法是后端的重要扩展。单独使用雷达时，呼吸率预测容易受到姿态变化和反射影响；单独使用呼噜时，不能判断呼吸是否中断。因此后端维护最近 30 到 60 秒时间窗，持续观察 `breath_rate`、`target_present`、`radar_online`、`snore_score`、`snore_level` 和 `dbfs`。若目标在床且雷达在线，呼吸率连续低于阈值或丢失超过约 10 秒，先形成候选窗口；若候选前后出现呼噜增强、声级反弹或呼吸率恢复波动，则提高置信度并生成 `suspected_apnea`。该规则不计算医学 AHI，只输出“疑似暂停次数”和看护提示。

Web 前端页面与交互设计

Web 前端是看护人员最主要的操作界面，采用 Vue 实现，并根据真实演示需要进行了多次简化。首页不再堆叠大段文字，而是以项目状态、入口卡片和真实设备照片为主，让评委或看护人员能快速理解系统构成。睡眠看护驾驶舱聚焦实时评分、设备在线、事件流、呼噜扰动地图和稳定性卡片；看护预警中心聚焦最高风险、SOS 处理、异常归因矩阵、照护动作队列和趋势小窗；环境分析页聚焦温度、湿度、环境声音、评分和建议。

前端展示原则是“信息少但关键”。紧急告警必须置顶并使用红色高优先级样式；普通趋势图不占据过长页面，避免用户需要频繁滚动；环境页将温湿度折线图与声级趋势图放在同一分析区域；呼噜强度不再孤立放在页面下方，而是与呼噜声浪或睡眠事件相关信息放在一起。所有页面字体相对放大，减少小字号解释性文字，按钮文案尽量短，避免在开发板小屏和 Web 页面上出现拥挤或截断。

本报告中的 Web 截图均来自项目实际前端运行页面，而不是重新绘制的概念图。截图时临时提供了后端接口数据，使页面能够展示评分、SOS、温湿度、呼噜分贝和事件流；页面结构、样式、侧边栏、卡片和图表均来自项目自己的 Vue 前端代码。这样文档既能体现系统真实界面，也便于后续替换为真实联调现场数据截图。

Android 监护人 App 设计

Android App 的定位是监护人远程报警终端，而不是替代 Web 驾驶舱。第一版采用 Kotlin 原生 Android 项目，用户在手机上填写后端地址，例如 `http://192.168.31.236:8081`，App 将地址保存到 `SharedPreferences`。点击开始监听后，前台服务常驻通知栏，每 5 秒轮询 `/sleep/overview?mode=live&seconds=1800`。当返回数据中存在 active critical 事件，尤其是 `emergency_voice`、`xiaozhi_imu_board` 或 `suspected_apnea`，App 弹出高优先级本地通知并播放提示音/震动。

这种后端绑定方式比单纯蓝牙直连更适合远程报警。蓝牙适合近场查看雷达摘要数据，但监护人手机可能不在床旁，且 Android 后台对蓝牙连接和扫描都有权限限制；后端轮询只要求手机能访问电脑或公网服务器，因此更适合课程演示和真实看护流程。后续如果需要近场调试，App 可以保留蓝牙雷达卡片；但 SOS、疑似呼吸暂停、温湿度、呼噜分数和分贝仍然建议通过后端汇聚后统一展示。

App 还实现了事件去重和处理入口。每个事件根据 `eventID` 或 `fingerprint` 生成本地记录，同一个事件不会反复弹窗；点击详情页的“已处理”后，App 调用 `/emergency/resolve`，并把 `source` 标记为 `android_guardian_app`。这样当手机端完成处理后，Web 预警中心和开发板的紧急状态也能同步解除，避免出现“手机已处理但前端仍报警”的割裂体验。

真实联调与测试方法

真实环境测试遵循先基础网络、再单板功能、最后全链路闭环的顺序。基础网络阶段先确认电脑 IP 为 `192.168.31.236`，后端监听 `8081` 端口，防火墙允许局域网访问；随后分别启动后端、前端和 Android App，验证 `/status` 与 `/sleep/overview` 能返回数据。单板阶段先测试 M55 的 Wi-Fi 保存、`backend_cfg_status`、`flash_test`、屏幕中文显示、用户按钮、小智语音、呼噜心跳、环境心跳和 IMU 状态；再测试雷达板 Wi-Fi、UDP 发送、心率/呼吸率/距离更新；最后测试 AHT20 初始化和温湿度变化。

全链路测试重点覆盖三类场景。第一类是主动求助：用户说“救命”或“需要帮助”，M55 屏幕进入 SOS，本地报警响起，后端生成 `emergency_voice critical` 事件，前端预警中心置顶，Android 手机 5 秒内通知，点击开发板或 App 解除后前端同步恢复。第二类是无法呼救：轻微打翻开发板或执行 `imu_fall_test`，系统以设备跌落为原因触发 SOS，并显示同样的解除按钮。第三类是疑似呼吸暂停：在雷达低呼吸窗口附近出现呼噜增强或声级反弹，后端生成 `suspected_apnea`，前端和 App 显示辅助风险提示，但不把它称为医学诊断。

稳定性测试关注线程、内存和状态一致性。开发板侧需要连续快速点击按钮、连续启停呼噜守护、连续触发并解除 SOS、播放 TTS/闹钟/报警时观察呼噜误报；后端侧需要重启服务后前端和 App 能恢复；雷达侧需要移动和静止切换后发送状态能恢复；前端侧需要长时间打开后图表持续刷新，不出现设备在线状态互相矛盾。通过这些测试，系统从“单功能可用”进一步接近“多功能同时运行可演示”。

安全性、可靠性与边界说明

本作品的安全策略以“及时提示、人工确认、避免自动高风险动作”为原则。系统 v1 不自动拨打电话、不自动发送短信，也不做医学诊断结论，避免误报造成不必要干扰。紧急事件只提醒看护人员立即确认床旁情况，并在提示词中要求小智语音用镇定、简短的语言建议用户呼叫身边人员或在紧急情况下拨打当地急救电话，不说“一定没事”，也不进行医学判断。

可靠性方面，系统尽量避免单点数据直接触发复杂结论。设备在线使用心跳时间判断；紧急状态使用 active/resolved 状态区分；呼噜和雷达融合使用时间窗和去重，避免同一段异常反复刷屏；开发板解除按钮既停止本地报警，也向后端同步处理结果；Android App 使用事件指纹去重。对于 Flash、Wi-Fi 配置和传感器初始化等嵌入式常见问题，项目加入了 flash_test、wifi_cfg_status、backend_cfg_status 和 IMU 状态命令，便于现场定位问题。

项目边界也需要明确：心率、呼吸率和疑似呼吸暂停结果受到雷达摆放、人体姿态、环境反射、模型样本和麦克风位置影响，不能替代医疗级睡眠监测、血氧仪或医院多导睡眠检查。系统更适合作为课程设计和看护辅助样机，用于展示 PSOC Edge/Edgi Talk、RT-Thread、AIROC Wi-Fi、边缘 AI 和多传感器融合在夜间安全场景中的工程应用价值。

核心算法与模块细化说明

雷达生命体征估计的输入是雷达板发送到后端的原始或半处理帧。后端处理时首先关注目标距离和回波稳定性，只有当目标存在且距离位于合理床旁范围时，心率和呼吸率才进入有效窗口。呼吸信号通常比心跳信号幅度更大，但也更容易受到翻身、离床、床被遮挡和板子移动影响；心率信号幅度更小，对噪声和距离更敏感。因此前端展示时不把瞬时值作为唯一依据，而是结合在线状态、趋势连续性和最近更新时间显示。

呼吸暂停融合逻辑的设计目标是“演示可信、误报可控”。当雷达短时间给出极低呼吸率时，系统不会立即报警，而是进入候选状态并等待持续时间达到阈值；如果候选状态结束后呼吸率恢复，同时呼噜强度或环境声级出现明显变化，后端才认为有更强的融合证据。这样的过程模拟了睡眠呼吸暂停中“呼吸减弱或停止—恢复性呼吸或声音变化”的可观察现象，但仍然只输出疑似提示。

后端对疑似呼吸暂停事件加入去重窗口。若同一段低呼吸窗口持续更新，系统只更新内部状态，不会每秒生成一条新事件；窗口结束或超过冷却时间后，才允许生成下一条事件。这个设计可以让预警中心保持可读性，避免评委或看护人员看到大量重复卡片。事件 details 中保留开始时间、结束时间、持续秒数、最低呼吸率、最大呼噜强度、最大分贝和置信度，便于后续复盘。

呼噜检测在边缘端完成，原因有三点：第一，长期上传原始音频会占用网络并带来隐私压力；第二，M55 具备边缘 AI 与 DSP/NPU 相关资源，适合运行轻量模型；第三，后端和前端只需要 snore_score、dbfs 和 detected 等摘要特征即可完成趋势展示和融合判断。模型量化为 int8 后，参数以头文件形式编译进固件，减少文件系统依赖，也避免运行时模型加载失败影响演示。

音频窗口处理采用固定采样率和固定窗口长度，使模型输入稳定。采集中心读取双声道 PCM 后，呼噜检测消费者取单声道数据写入环形缓冲；当缓冲达到 2 秒窗口时执行一次推理，并保留最近推理时间、原始输出、归一化分数、分贝估计和是否命中的状态。若网络暂时不可用，呼噜检测仍可在本地显示推理结果；网络恢复后继续上报心跳。

小智语音服务与呼噜守护同时运行时，状态切换必须谨慎。待机状态下唤醒词和呼噜检测常驻；按下用户按钮或唤醒词触发后，小智进入聆听并开始上传语音，同时呼噜检测继续运行；对话结束后关闭语音上传，恢复唤醒词推理。用户暂停呼噜守护时，只关闭呼噜消费者，不关闭麦克风和小智语音。这样既能保障“救命”等关键词随时可用，也能满足睡眠呼噜守护的连续性。

SOS 状态机采用统一入口和统一出口。入口包括 STT 关键词命中、IMU 跌落触发以及调试命令；进入状态后，UI 显示红色 SOS 页面、报警声开始播放、后端收到 /emergency 事件、前端和 App 显示 critical 告警。出口包括开发板解除按钮、Web 处理按钮和 App 已处理按钮；解除后本地停止报警，UI 回到主页，后端事件状态改为 resolved。统一状态机减少了“语音 SOS 能解除但 IMU SOS 不能解除”这类不一致问题。

报警声和闹钟声都使用设备扬声器播放，但两者在语义上不同。报警声用于紧急状态，音量更高、持续更久，并配合 SOS 页面；闹钟声用于日常提醒，到点后进入闹钟解除界面，用户点击关闭后返回主页。为避免声音播放阻塞系统，播放过程不应长时间占用 UI 线程；同时，播放状态会通知呼噜检测抑制误报，避免把自己的声音识别为异常声音。

环境分析算法不追求复杂模型，而是使用可解释评分。温度建议范围设为约 18 到 24 摄氏度，湿度建议范围设为约 40% 到 60%，环境声音使用 dBFS 相对值判断安静程度。三个分项分别计算适宜度，再形成环境总评分。前端显示建议时使用“偏热、偏冷、偏干、偏湿、噪声偏高”等直观描述，使看护人员可以采取开窗、加湿、调整空调、降低噪声等简单动作。

设备状态一致性是前端体验的重要部分。早期页面曾出现 Edgi 总状态在线，但呼噜板和环境传感器显示离线的问题，原因是后端把不同功能模块的心跳分开统计，而前端又把它们当成独立设备。当前设计将 Edgi E84 看作一块综合开发板，同时保留呼噜、环境和语音服务的子状态。这样用户能看懂“开发板在线，但呼噜暂停”“开发板在线，环境传感器异常”“开发板处于紧急状态”等细分情况。

数据字段与页面映射

睡眠看护驾驶舱主要消费 /sleep/overview。score 字段用于中央评分环，stats 字段用于数据点、疑似暂停次数和统计摘要，devices 字段用于雷达板、Edgi E84 和紧急状态胶囊，events 字段用于夜间守护事件流，heatmap 字段用于呼噜扰动地图，stability_cards 字段用于在线率、心跳稳定性和环境舒适度卡片。前端不直接计算复杂结论，只负责把后端输出以更易读的方式呈现。

看护预警中心同时读取 /sleep/overview、/timeline 和 /status。/status 提供当前心率、呼吸率、设备在线、板子静止、温湿度、呼噜状态和 active emergency；/timeline 提供最近 30 分钟趋势，用于心率、呼吸、呼噜、温度和湿度曲线；/sleep/overview 提供事件列表和评分。预警中心根据这些数据生成异常归因矩阵和照护动作队列，例如紧急求助时置顶“立即确认床旁情况”，疑似呼吸暂停时建议“确认呼吸状态、调整睡姿、检查雷达位置”。

环境分析页主要读取 /status 和 /timeline。当前温度、湿度和分贝来自 /status 的最新值，温湿度折线图和環境声音趋势来自 /timeline。若 AHT20 长时间没有心跳，页面显示环境传感器离线；若 Edgi 在线但 sensor_ok 为 false，则显示传感器异常而不是整板离线。这样的区分可以帮助测试人员判断是网络问题、传感器初始化问题还是单纯没有数据。

Android App 读取的数据与 Web 保持一致，但展示粒度更简洁。App 首页显示后端地址、监听状态、当前 active 报警、温湿度、呼噜分数、分贝、心率和呼吸率；通知逻辑只关注 active critical 或高风险 suspected_apnea。这样手机端不会像 Web 一样承载大量图表，而是成为监护人的提醒终端。

历史数据页用于后续长期记录扩展。当前项目已经具备 timeline 和 sleep_events 的数据结构，后续可以把内存状态写入数据库，按日期回放睡眠评分、疑似暂停、呼噜强度、环境评分和事件处理记录。比赛提交文档中重点展示实时闭环页面，是因为当前演示以真实硬件现场运行和即时告警为主。

部署流程与现场演示脚本

现场演示前需要先确认电脑、开发板和手机处于同一网络，电脑 IP 为 192.168.31.236。第一步启动后端：在项目根目录运行 python backend\realtime_radar_processing.py，确认控制台打印 /status、/sleep/overview 和 UDP 监听信息。第二步启动前端：进入 frontend 目录运行 npm run dev，在浏览器打开睡眠看护驾驶舱和预警中心。第三步打开 Android App，填写 http://192.168.31.236:8081，点击开始监听。

开发板侧先测试 Edgi Talk M55。串口执行 backend_cfg_status 确认目标地址为 192.168.31.236:8081，如不是则执行 backend_cfg_set 192.168.31.236 8081。随后确认 Wi-Fi 已自动连接，屏幕主页显示正常，呼噜守护自动启动，环境心跳正常上报。若首次配网，则连接开发板 AP，在浏览器打开配网页输入 Wi-Fi 信息，保存后重启验证。

雷达板演示时先确认 Wi-Fi 连接成功并打印 IP，后端收到启用命令后开始接收雷达数据。前端应显示雷达板在线、心率和呼吸率有值、目标距离正常。如果启用了静止门控，移动雷达板时前端应提示板子未静止；若测试阶段关闭门控，则重点观察 UDP 数据是否连续。为了让数据更稳定，雷达板应固定朝向人体胸腹区域，距离保持在可检测范围内。

演示脚本可以按“平稳运行—普通呼噜—疑似暂停—紧急求助—处理解除”的顺序进行。先展示首页和驾驶舱，说明雷达、呼噜、环境和设备状态；再播放或制造呼噜样本，观察呼噜分数和事件流；

随后通过构造低呼吸窗口和声音变化展示 suspected_apnea；最后说出“救命”或执行 IMU 测试触发 SOS，让 Web 和 Android 同时报警，并现场点击解除按钮完成闭环。

如果现场网络不稳定，应优先保证后端和前端在电脑本机可访问，再排查开发板能否 ping 通电脑 IP、电脑防火墙是否允许 8081、开发板保存的后端 IP 是否过期、手机是否与电脑处于同一网段。对比赛演示而言，最重要的是让关键闭环稳定跑通，而不是同时展示所有调试功能。

测试覆盖与结果记录

基础接口测试包括 GET /status、GET /timeline、GET /sleep/overview、POST /emergency 和 POST /emergency/resolve。/status 应能返回雷达、Edgi、呼噜、环境、语音、紧急状态和最近心跳时间；/timeline 应能返回趋势数组；/sleep/overview 应包含 score、events、devices、heatmap 和 stability_cards；/emergency 应创建 critical 事件；/emergency/resolve 应把 active emergency 改为 resolved。

M55 固件测试覆盖 Wi-Fi 保存、Flash 读写、后端 IP 配置、中文 UI、按钮连续点击、小智聆听、STT/TTS、关键词 SOS、IMU SOS、呼噜自动启动、暂停/恢复呼噜、闹钟响铃与解除、环境心跳和长时间运行。特别需要测试“连续按两下左侧按钮不会锁死”“SOS 解除后回到主页”“报警持续时间足够明显”“呼噜守护与小智语音同时可用”这些真实调试中暴露过的问题。

雷达板测试覆盖 Wi-Fi 连接、UDP 发送、心率/呼吸率/距离更新、移动/静止切换、LED 指示、传感器异常恢复和前端在线状态。若雷达呼吸率不稳定，应检查板子朝向、人体距离、周围反射物、是否有大幅体动，以及后端滤波窗口是否过短。对于疑似呼吸暂停，测试时应构造连续低呼吸窗口，而不是只改一个瞬时点。

Edgi Talk 的 M33/AHT20 环境采集测试覆盖传感器初始化、温湿度周期采样、共享内存更新、sensor_ok 状态、前端环境评分变化和异常恢复。实际测试中，温湿度变化较慢，因此可以通过手握传感器、靠近湿纸巾或改变空调环境观察趋势变化；但报告中不强调精确校准，而强调跨核数据链路和评分逻辑。

Android App 测试覆盖 APK 安装、后端地址保存、开始监听、后台和锁屏通知、同一事件去重、后端离线恢复、点击已处理后同步解除。由于 Android 13 及以上需要通知权限，首次启动时必须允许 POST_NOTIFICATIONS；前台服务通知栏显示“看护监听中”是后台轮询正常工作的标志。

稳定性回归测试包括连续运行 30 分钟到 2 小时，观察线程数量、堆内存、串口异常、麦克风抢占、网络请求失败和前端刷新。开发板侧重点看是否出现 rt_thread_delete 断言、malloc failed、WebSocket write busy 持续堆积或 Wi-Fi 配置丢失；后端侧重点看事件是否重复刷屏、设备是否误离线；前端侧重点看图表是否重绘正常，文字是否重叠。

作品完整度与比赛展示价值

从比赛评审角度看，本作品覆盖了英飞凌赛道关注的多个方向：PSOC Edge/Edgi Talk 边缘智能、RT-Thread 嵌入式任务调度、AIROC Wi-Fi 网络连接、毫米波雷达感知、边缘 AI 模型推理、多传感器融合、Web 可视化和移动端应用。作品不是单个 demo，而是把多个模块组合为一个可演示的看护系统，能够体现硬件、固件、算法、后端、前端和 App 的综合开发能力。

与传统养老看护项目相比，本作品将重点从“泛化看护”收敛到“夜间睡眠监测”。这个方向更适合非接触感知：用户睡眠时不方便佩戴设备，也不希望摄像头拍摄隐私画面；床旁雷达和麦克风可以在不接触身体的情况下观察呼吸、心率、呼噜和求助语音；温湿度和声音趋势又能解释睡眠环境是否舒适。聚焦睡眠场景后，系统功能之间的关系更清楚，演示链路也更完整。

作品的工程难点主要来自真实联调，而不是页面展示。音频共享、Flash 保存、Wi-Fi 自动连接、后端 IP 配置、中文乱码、按钮锁死、报警解除、呼噜误报、设备在线状态和 Android 后台通知都在调试过程中逐步解决。报告中保留这些设计说明，可以让评委看到项目并非只停留在方案设想，而是经历了真实开发板问题定位和迭代。

后续若继续完善，可把 M55、雷达和环境传感器做成一体化外壳，增加设备配置页和 OTA 更新；加入血氧或床垫压力传感器，提高呼吸暂停风险判断可信度；将后端部署到云服务器，并通过厂商推送替代手机轮询；积累真实夜间数据后训练更稳定的呼噜和呼吸异常模型。当前版本则优先保证比赛现场能稳定展示“采集—融合—告警—处理”的闭环。

第四部分 总结

本作品完成了从边缘硬件、后端融合、前端展示到移动端提醒的完整睡眠看护闭环。项目的核心价值不在于单一传感器精度，而在于多模态数据协同：雷达用于非接触生命体征，麦克风用于呼噜和求助语音，IMU 用于无法呼救时的设备跌落触发，AHT20 用于环境舒适度评估。通过后端统一状态层，系统把这些输入转化为看护人员容易理解和处理的事件流。

可扩展之处

后续可加入血氧、床垫压力或更稳定的雷达支架，提高疑似呼吸暂停判断的可靠性；可把 Android App 从轮询升级为公网推送，降低后台耗电；可设计一体化外壳和设备配置页，让后端地址、报警音量、闹钟和关键词维护更方便；也可增加长周期睡眠报告，按周或月统计呼噜、疑似暂停、环境评分和报警处理记录。

心得体会

开发过程中最大的收获是认识到真实嵌入式系统不是把算法跑通就结束。音频采集、线程生命周期、Flash 写入、Wi-Fi 配网、串口日志、前端在线状态和移动端后台通知都会影响最终演示效果。例如，早期语音服务和呼噜守护分别打开 mic0，导致抢占和线程断言；后来改为共享音频采集中心

后，系统才真正支持“呼噜守护常驻，同时小智能识别求助语音”。又如，紧急状态下报警声会干扰呼噜检测，因此需要在本地播放期间抑制呼噜误报。

系统设计也需要在“强告警”和“误报控制”之间保持谨慎。语音 SOS 和 IMU 跌落属于高风险事件，系统立即进入 critical；疑似呼吸暂停则采用融合规则，不把单点雷达异常或单独呼噜直接当作呼吸暂停。这种设计更符合看护辅助系统的定位：尽早提醒监护人关注，但不替代医学诊断。通过本项目，我们完成了边缘 AI、RT-Thread 多任务、真实硬件接口、Web 可视化和 Android 后台服务的综合实践，也更理解了从课程样机走向实际应用所需要的工程细节。

第五部分 参考文献

- [1] Infineon Technologies AG. PSOC Edge E84 Consumer Datasheet.
- [2] Infineon Technologies AG. CY8CKIT-062S2-AI PSoC 6 AI Evaluation Kit User Guide.
- [3] RT-Thread. RT-Thread Operating System Documentation.
- [4] Infineon Technologies AG. AIROC Wi-Fi/Bluetooth Technical Resources.
- [5] Zhao W, Tong G. A deep learning based heart rate estimation method for millimeter wave radar[J]. Measurement, 2025.
- [6] AASM. Sleep-related breathing disorders and sleep apnea clinical background materials.